

Trabajo Práctico 1

- Tobias Di Salvo
- Angie Oldano
- Ariel Paz

¿Qué hace el Ejercicio 1?

Algoritmo FIND-S

1. El algoritmo FIND-S comienza con la hipótesis más específica y la generaliza considerando sólo los ejemplos positivos.
2. Ignora los ejemplos negativos, asumiendo que el espacio de hipótesis contiene una hipótesis que describe el concepto objetivo verdadero c , y que el conjunto de entrenamiento no contiene errores.
3. Encuentra, dentro del espacio de hipótesis, la hipótesis h más específica que es consistente con los ejemplos de entrenamiento positivos.

La hipótesis más específica:

$$h = [\emptyset, \emptyset, \dots, \emptyset]$$

La hipótesis más general:

$$h = [?, ?, \dots, ?]$$

Implementación

```
1 import pandas as pd
2
3 df = pd.read_csv('prestamo.csv', encoding='latin-1')
4
5 # Filtrar por 50 años
6 df_50 = df[df['Edad'] == 50].copy()
7
8 columnas = ['Sexo', 'Mayor nivel educativo', 'Estado de vivienda', 'Préstamos previos impagos']
9 estado = 'Estado'
10
11 # Dividir en Entrenamiento (75%) y Prueba (25%)
12 limite = int(len(df_50) * 0.75)
13 entrenamiento = df_50.iloc[:limite]
14 prueba = df_50.iloc[limite:]
15
16 # Algoritmo FIND-S
17 def find_s(data, atributos, concepto):
18     h = ['0'] * len(atributos) # Hipótesis inicial
19     positivos = data[data[concepto] == 'OTORGADO']
20     for j, fila in positivos.iterrows():
21         for i, col in enumerate(atributos):
22             valor_actual = str(fila[col])
23             if h[i] == '0':
24                 h[i] = valor_actual
25             elif h[i] != valor_actual:
26                 h[i] = '?'
27     return h
28
29 hipotesis = find_s(entrenamiento, columnas, estado)
30 print(f"Hipótesis obtenida: {hipotesis}")
31
```

Post Algoritmo FIND-S

Predicción

1. Se descarta los atributos generalizados (?) para enfocarse únicamente en los atributos determinantes.
2. Si el dato del conjunto de prueba contradice un valor específico de la hipótesis, el algoritmo rechaza la instancia inmediatamente
3. Find-S supone que solo lo que encaja perfectamente en su patrón específico puede ser un ejemplo positivo.

Implementación

```
32
33 # Algoritmo de predicción
34 def predecir(hipotesis, registro, atributos):
35     for i, valor in enumerate(hipotesis):
36         if valor != '?' and valor != registro[atributos[i]]:
37             return 'RECHAZADO'
38     return 'OTORGADO'
39
40
41 # Predicción en el conjunto de prueba
42 aciertos = 0
43 no_aciertos = 0
44 for i, registro in prueba.iterrows():
45     prediccion = predecir(hipotesis, registro, columnas)
46     es_correcto = prediccion == registro[estado]
47     if es_correcto:
48         aciertos += 1
49     else:
50         no_aciertos += 1
51
52
53     print(f"Real: {registro[estado]} | Predicción: {prediccion} | {'OK' if es_correcto else 'ERROR'}")
54 print(aciertos)
55 print(no_aciertos)
56
```

Resultados - FIND-S y Predicción

Algoritmo FIND-S:

La hipótesis inicial:

$h = [0, 0, 0, 0]$

La hipótesis obtenida:

$h = ['?', '?', '?', 'NO']$

Predicción:

Cantidad de aciertos:

10

Cantidad de no aciertos:

4

¿Qué hace el Ejercicio 2?

Métricas

Se busca obtener una serie de métricas para poder evaluar el desempeño del modelo construido previamente.

Orientamos el código para hacerlo reutilizable para futuros usos.

Matriz de confusión y métricas

Matriz

Matriz de confusión

Relaciona los casos positivos y negativos predichos con los reales

		Predicciones	
		Positivos	Negativos
Reales	Positivos	Verdaderos Positivos (TP)	Falsos Negativos (FN)
	Negativos	Falsos Positivos (FP)	Verdaderos Negativos (FN)

Métricas de evaluación

Accuracy $(TP+TN) / total$

Recall TP / P

Specificity TN / N

Precision $TP / TP + FP$

F1 Score $2 * precision * recall / precision + recall$

True Positive Rate (TRP) TP / P

False Positive Rate (FPR) FP / N

Resultados — Modelo del ejercicio 1

Matriz

Matriz de confusión

Para el ejemplo realizado

		Predicciones	
		Positivos	Negativos
Reales	Positivos	4	0
	Negativos	4	6

Métricas de evaluación

Accuracy

0.7142...

Recall

1.0

Specificity

0.6

Precision

0.5

F1 Score

0.6666...

True Positive Rate (TRP)

1.0

False Positive Rate (FPR)

0.4

Implementación

```
##### Métricas #####

# Calculamos los 4 valores posibles de los datos: True Positive (TP), False Positive (FP), True Negative (TN), False Negative (FN)
# para un vector de predicciones y uno de esperados. OTORGADO == Positive, RECHAZADO == Negative
def calcularPositivosYnegativos(predicciones, esperados):
    TP = 0
    FP = 0
    TN = 0
    FN = 0
    # Recorremos el vector de predicciones y comparamos el resultado con el esperado
    for i in range(len(predicciones)):
        if predicciones[i] == 'OTORGADO': # Predicciones POSITIVAS
            if esperados[i] == 'OTORGADO':
                TP += 1
            else:
                FP += 1
        else: # Predicciones NEGATIVAS
            if esperados[i] == 'RECHAZADO':
                TN += 1
            else:
                FN += 1
    return TP,FP,TN,FN

# Creamos los conjuntos de predicciones y de esperados
conj_predicciones = []
conj_esperados = []

for i, registro in prueba.iterrows():
    conj_predicciones.append(predicir(hipotesis, registro, columnas))
    conj_esperados.append(registro[estado])
```

Implementación

```
# Calculamos sus variables
TP, FP, TN, FN = calcularPositivosYnegativos(conj_predicciones, conj_esperados)

# Matriz de confusión para un conjunto de predicciones y esperados
def matrizDeConfusion(predicciones, esperados):
    tp, fp, tn, fn = calcularPositivosYnegativos(predicciones, esperados)
    return [[tp, fn], [fp, tn]]

def accuracy(tp, fp, tn, fn):
    return (tp + tn) / (tp + tn + fp + fn)

def recall(tp, fn):
    return tp / (tp + fn)

def especificidad(fp, tn):
    return tn / (tn + fp)

def precision(tp, fp):
    return tp / (tp + fp)

def f1_score(tp, fp, fn):
    return 2 * (precision(tp, fp) * recall(tp, fn)) / (precision(tp, fp) + recall(tp, fn))

def tpr_fpr(tp, fp, tn, fn):
    return tp / (tp + fn), fp / (tn + fp)
```

¿Qué hace el Ejercicio 3?

Predecir si un préstamo es OTORGADO o RECHAZADO para personas de 40–45 años, usando solo atributos categóricos.



Datos y Entrenamiento

Conjunto de datos

Edad filtrada	40 – 45 años
Total ejemplos (N)	1.426
Entrenamiento (80%)	1.140
Prueba (20%)	286
Clases (K)	OTORGADO / RECHAZADO
Probabilidad de ser otorgado	20,4 %
Probabilidad de ser rechazado	79,6 %

Atributos categóricos

Sexo (Femenino / Masculino)

Mayor nivel educativo

Estado de vivienda

Préstamos previos impagos (Sí / No)

Suavizado de Laplace ($I = 1$)

Evita que una probabilidad sea 0 cuando un valor no aparece en entrenamiento.

Implementación

```
130
131 df = pd.read_csv('prestamo.csv', encoding='latin-1')
132
133 # Filtrar por edad entre 40 y 45
134 df_filtrado = df[(df['Edad'] >= 40) & (df['Edad'] <= 45)].copy()
135
136 columnas = ['Sexo', 'Mayor nivel educativo', 'Estado de vivienda', 'Préstamos previos impagos']
137 estado = 'Estado'
138
139 # Dividir en Entrenamiento (80%) y Prueba (20%)
140 limite = int(len(df_filtrado) * 0.80) # saco la cantidad de el %80 de la cantidad de registros
141 entrenamiento = df_filtrado.iloc[:limite] # tomo los primeros registros para entrenamiento
142 prueba = df_filtrado.iloc[limite:] # y los demas hasta el final para prueba
143
144 clases = entrenamiento[estado].unique()
145 N = len(entrenamiento) # total de ejemplos
146 d = len(columnas) # cantidad de atributos
147 l = 1 # suavizado de Laplace
148 K = len(clases) # cantidad de clases
149
150 # probabilidades de cada clase
151 pi = {}
152 for c in clases:
153     #  $\pi_k = (\#\{Y = C_k\} + 1) / (N + 1 * K)$ 
154     pi[c] = (len(entrenamiento[entrenamiento[estado] == c]) + 1) / (N + 1 * K)
155
```

Clasificación del Conjunto de Prueba

Resultado del modelo para cada ejemplo: se muestra clase real vs. clase predicha y los scores internos.

Total en prueba

286

Correctos 

216

Incorrectos 

70

Ej.	Real	Predicción	Score OTORGADO	Score RECHAZADO	¿Correcto?
224	OTORGADO	OTORGADO	0.024605	0.019562	
226	OTORGADO	OTORGADO	0.031294	0.027782	
1	RECHAZADO	RECHAZADO	0.000009	0.008949	
2	RECHAZADO	RECHAZADO	0.000119	0.037876	
225	OTORGADO	RECHAZADO	0.008784	0.031189	
233	OTORGADO	RECHAZADO	0.001122	0.003456	
3	RECHAZADO	OTORGADO	0.021871	0.018025	
9	RECHAZADO	OTORGADO	0.003554	0.002837	

Implementación

```
for col in columnas:
    x[col] = {}
    valores_posibles = entrenamiento[col].unique() # Mj valores que puede tomar cada atributo
    Mj = len(valores_posibles)

    for val in valores_posibles:
        x[col][val] = {}
        for c in clases:
            filas_clase = entrenamiento[entrenamiento[estado] == c]
            num = len(filas_clase[filas_clase[col] == val]) + 1
            denom = len(filas_clase) + 1 * Mj
            x[col][val][c] = num / denom

def calculoBayes(fila, pi, x, clases, columnas):
    scores = {}
    for c in clases:
        score = pi[c]
        for col in columnas:
            val = fila[col]
            score *= x[col][val][c]
        scores[c] = score
    return max(scores, key=scores.get), scores
```

$$Y \leftarrow \underset{\{C_k: k=1,2,\dots,K\}}{\operatorname{argmax}} \left\{ P(Y = C_k) \prod_{j=1}^d P(X_j = x_j | Y = C_k) \right\}$$

Resultados — Matriz de Confusión y Métricas

Matriz de Confusión (conjunto de prueba, n = 286)

Matriz de confusión

Para el ejemplo realizado

	Pred. OTORGADO	Pred. RECHAZADO
Real OTORGADO	VP = 33	FN = 30
Real RECHAZADO	FP = 40	VN = 183

Métricas de Evaluación

Accuracy

$(VP+VN) / total$

75.52%

Precisión

$VP / (VP+FP)$

45.21%

Recall

$VP / (VP+FN)$

52.38%

F1-score

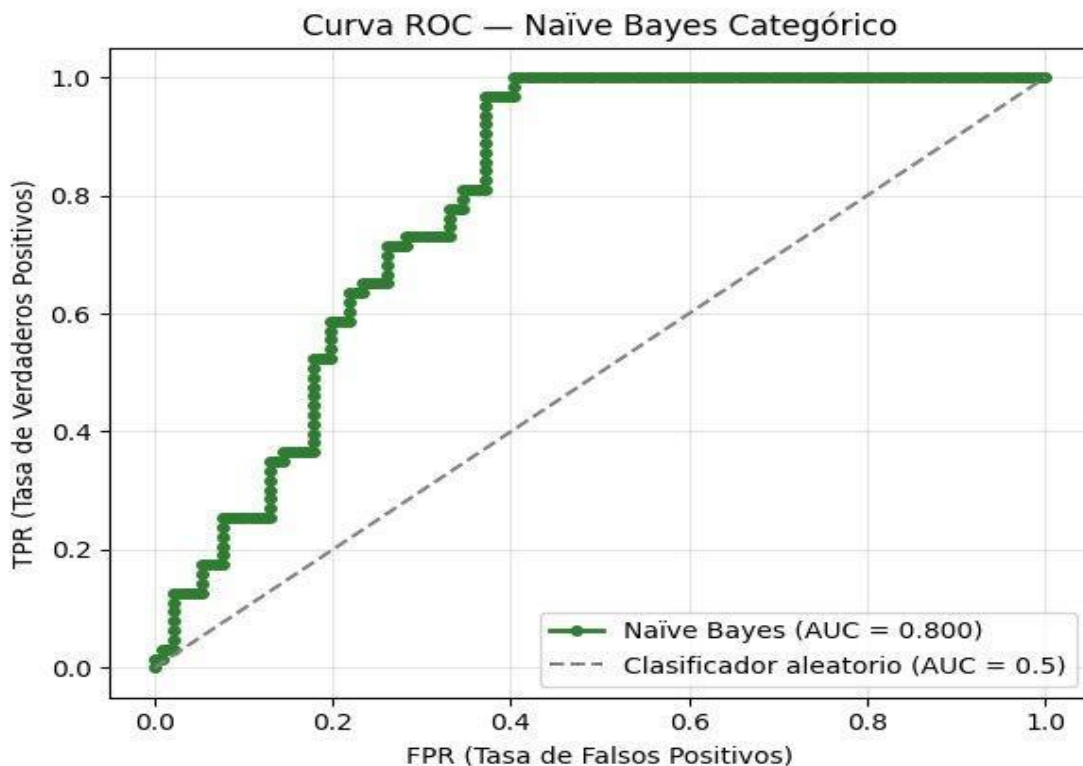
$2 \cdot Prec \cdot Rec / (Prec+Rec)$

48.53%

Implementación

```
185 y_pred = []
186 y_prob = []
187 y_real = []
188
189 clase_rech = clases[0]
190 clase_otorg = clases[1]
191
192 for _, fila in prueba.iterrows():
193     pred, scores = calculoBayes(fila, pi, x, clases, columnas)
194     y_pred.append(pred)
195     y_real.append(fila[estado])
196
197     # P1 / (P1 + P2)
198     total = sum(scores.values())
199     y_prob.append(scores[clase_otorg] / total if total > 0 else 0)
200
201 TP, FP, TN, FN = calcularPositivosYnegativos(y_pred, y_real)
202
203 print(f"\nClase: {clase_otorg}")
204 print(f"\nMatriz de confusión:")
205 print(f"                Pred {clase_otorg}    Pred {clase_rech}")
206 print(f"Real {clase_otorg}      TP={TP}          FN={FN}")
207 print(f"Real {clase_rech}      FP={FP}          TN={TN}")
208
209
210 print(f"\nAccuracy : {accuracy(TP,FP,TN,FN):.4f}")
211 print(f"Precisión : {precision(TP,FP):.4f}")
212 print(f"Recall : {recall(TP,FN):.4f}")
213 print(f"F1-score : {f1_score(TP,FP,FN):.4f}")
```

Curva ROC y AUC



¿Cómo se construye?

- 1 Calcular tasa de verdaderos positivos (TPR) y tasa de falsos positivos (FPR) en cada paso.
- 2 Graficar cada par (FPR, TPR) como un punto.
- 3 Conectar los puntos → curva ROC.

AUC = 0.800

Clasificador razonable ($0.5 < \text{AUC} \leq 1$)

Implementación

```
# GRAFICO Curva ROC
# score y clase real en una lista de pares ordenados
pares = sorted(zip(y_prob, y_real), reverse=True)

total_pos = TP + FN    # P = total positivos reales
total_neg = FP + TN    # N = total negativos reales

puntos_fpr = [0]
puntos_tpr = [0]

tp = 0
fp = 0

for score, real in pares:
    if real == clase_otorg:
        tp += 1    # era positivo y ahora lo clasificamos como positivo → VP
    else:
        fp += 1    # era negativo pero lo clasificamos como positivo → FP
    tpr = tp / total_pos    # TPR = VP / P
    fpr = fp / total_neg    # FPR = FP / N
    puntos_tpr.append(tpr)
    puntos_fpr.append(fpr)
```

Implementación

```
245
246 auc_val = 0
247 for i in range(len(puntos_fpr) - 1):
248     base = puntos_fpr[i+1] - puntos_fpr[i]      # ancho del trapecio
249     altura = (puntos_tpr[i+1] + puntos_tpr[i]) / 2 # altura promedio
250     auc_val += base * altura
251
252 print(f"\nAUC = {auc_val:.4f}")
253
254 plt.figure(figsize=(6, 5))
255
256 # Curva ROC del modelo
257 plt.plot(puntos_fpr, puntos_tpr, color='█'#2E7D32', lw=2,
258 | | | | marker='o', markersize=4, label=f'Naïve Bayes (AUC = {auc_val:.3f})')
259
260 # Línea del clasificador aleatorio (TPR = FPR)
261 plt.plot([0, 1], [0, 1], color='gray', lw=1.5,
262 | | | | linestyle='--', label='Clasificador aleatorio (AUC = 0.5)')
263
264 plt.xlabel("FPR (Tasa de Falsos Positivos)")
265 plt.ylabel("TPR (Tasa de Verdaderos Positivos)")
266 plt.title("Curva ROC – Naïve Bayes Categórico")
267 plt.legend(loc='lower right')
268 plt.grid(alpha=0.3)
269 plt.tight_layout()
270 plt.show()
```

¿Preguntas?

Gracias por escucharnos.

